

Introduction

Doom is a documentation development tool designed for internal use at Alauda, built on top of [rspress](#)[↗]. It provides users with a rich set of built-in plugins for an out-of-the-box experience.

TOC

Core Capabilities

Based on Markdown and its extension MDX

Get Started

Getting Started

Usage

Core Capabilities

- Automatically generates a configurable weight (order) left sidebar
- Full-text static document search
- Multilingual support

Based on Markdown and its extension MDX

MDX is a powerful content development approach. You can write Markdown files as you normally would while also using React components within the Markdown content:



```
// docs/index.mdx
import { CustomComponent } from './custom';

# Hello World

<CustomComponent />
```

For more details, you can refer to the [“Using MDX” documentation](#).

Get Started

Let's [get started quickly](#) with Doom!

Getting Started

Getting Started

Creating a Project

CLI Tool

Usage

Configuration

Configure the `doom` documentation tool

Configuration File

Basic Configuration

API Documentation Configuration

Permission Documentation Configuration

Reference Documentation Configuration

Release Notes Configuration

Sidebar Configuration

Internal Documentation Routes Configuration

Only Include Documentation Routes Configuration

Syntax Highlighting Plugin Configuration

`sites.yaml` Configuration

Translation Configuration

Editing Documentation in Code Repository

Documentation Export Configuration

Documentation Lint Configuration

Algolia Search Configuration

Sitemap Configuration

Convention

Based on the principle of "convention over configuration", we agree on the organization of documents to automatically generate the left sidebar and related content.

Directory Structure

Metadata

Sorting

Preview

Markdown

Callouts

Mermaid

MDX

Using MDX enables dynamic content display and content reuse

rspress Components

doom Components

Custom Component Reuse

Internationalization

Using Internationalized Text in Reusable Components

`i18n.json`

`.ts/.tsx`

`.mdx`

API Documentation

Advanced APIs

CRD

Common References

Specifying OpenAPI Path

Permission Description Document

`props`

Example

Referencing Documents

Document Reference Configuration

Deployment

After the documentation project development is completed, we can deploy the project to the ACP

Getting Started

TOC

Creating a Project

CLI Tool

Starting the Development Server

Production Build

Local Preview

Using Scaffolding Templates

Translating Documentation

Exporting PDF

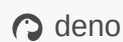
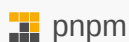
Documentation Linting

Creating a Project

First, you can create a new directory with the following command:

```
mkdir my-docs && cd my-docs
```

Run `npm init -y` to initialize a project. You can install doom using npm, yarn, or pnpm:



```
npm install -D @alauda/doom typescript
```

Then create files with the following commands:

```
# Create docs directories, default supports bilingual Chinese and English
mkdir docs/en && echo '# Hello World' > docs/en/index.md
mkdir docs/zh && echo '# 你好' > docs/zh/index.md
```

Add the following scripts to your `package.json`:

```
{
  "scripts": {
    "dev": "doom dev",
    "build": "doom build",
    "new": "doom new",
    "serve": "doom serve",
    "translate": "doom translate",
    "export": "doom export"
  }
}
```

Then initialize a configuration file `doom.config.yml`:

```
title: My Docs
```

Also create a `tsconfig.json` file with the following content:

```
{
  "compilerOptions": {
    "jsx": "react-jsx",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "resolveJsonModule": true,
    "skipLibCheck": true,
    "strict": true,
    "target": "ESNext",
  },
  "mdx": {
    "checkMdx": true,
  },
}
```

Finally, create a `global.d.ts` file with the following content:

```
/// <reference types="@alauda/doom/runtime" />
```

This allows you to use doom's global components in `.mdx` files with type safety.

CLI Tool


```
doom -h
```

```
# output
```

```
Usage: doom [options] [command]
```

```
Doctor Doom making docs.
```

```
Options:
```

```
  -V, --version                output the version number
  -c, --config <config>       Specify the path to the config file
  -v <version>                 Specify the version of the document
                                ation, can also be `unversioned` or `unversioned-x.y`
  -b, --base <base>           Override the base of the documentat
                                ion
  -p, --prefix <prefix>       Specify the prefix of the documenta
                                tion base
  -f, --force [boolean]        Force to
                                1. fetch latest reference remotes o
                                r scaffolding templates, otherwise use local cache
                                2. translate ignore hash equality c
                                heck and original text (default: false)
  -i, --ignore [boolean]       Ignore internal routes (default: fa
                                lse)
  -d, --download [boolean]     Display download pdf link on nav ba
                                r (default: false)
  -e, --export [boolean]       Run or build in exporting PDF mode,
                                `apis/**` and `*/apis/**` routes will be ignored automatically (default:
                                false)
  -I, --include <language...> Include **only** the specific langu
                                age(s), `en ru` for example
  -E, --exclude <language...> Include all languages except the sp
                                ecific language(s), `ru` for example
  -o, --out-dir <path>         Override the `outDir` defined in th
                                e config file or the default `dist/{base}/{version}`, the resulting path
                                will be `dist/{outDir}/{version}`
  -r, --redirect <enum>        Whether to redirect to the locale c
                                losest to `navigator.language` when the user visits the site, could be `a
                                uto`, `never` or `only-default-lang` (default: "only-default-lang")
  -R, --edit-repo [boolean|url|lang] Whether to enable or override the `
                                editRepoBaseUrl` config feature, `https://github.com/` prefix could be om
                                itted; if a language code is provided, all other languages will use the s
                                ame repo but with the language code as sub-path (default: false)
  -a, --algolia [boolean|alauda] Whether to enable or use the alauda
```

```
(docs.alauda.io) preset for Algolia search (default: false)
  -S, --site-url Whether to enable the siteUrl for s
itemap generation (default: false)
  -n, --no-open Do not open the browser after start
ing the server
  --lang <language> Specify the language of the documen
tation
  -h, --help display help for command

Commands:
  dev [options] [root] Start the development server
  build [root] Build the documentation
  preview|serve [options] [root] Preview the built documentation
  new [template] Generate scaffolding from templates
  translate [options] [root] Translate the documentation
  export [options] [root] Export the documentation as PDF, `a
pis/**` and `*/apis/**` routes will be ignored automatically
  lint [options] [root] Lint the documentation
  help [command] display help for command
```

For more configuration, please refer to [Configuration](#)

Starting the Development Server

Run `yarn dev` to start the development server, the browser will automatically open the documentation homepage.

```
doom dev -h
```

```
# output
```

```
Usage: doom dev [options] [root]
```

```
Start the development server
```

```
Arguments:
```

```
  root                                Root directory of the documentation
```

```
Options:
```

```
  -H, --host [host]                  Dev server host name
  -P, --port [port]                  Dev server port number
  -l, --lazy [boolean]               Whether to enable `lazyCompilation` which could improve the compilation performance (default: true)
  --no-lazy                          Do not enable `lazyCompilation`
  -h, --help                          display help for command
```

Production Build

Run `yarn build` to build the production code. After building, static files will be generated in the `dist` directory.

Local Preview

Run `yarn serve` to preview the built static files. Note that if you used `-b` or `-p` parameters during build, you need to use the same `-b` and `-p` parameters during preview.

Using Scaffolding Templates

Run `yarn new` to generate projects, modules, or documentation using scaffolding templates.

Translating Documentation

```
doom translate -h
```

```
# output
```

```
Usage: doom translate [options] [root]
```

```
Translate the documentation
```

```
Arguments:
```

```
  root                                Root directory of the documentation
```

```
Options:
```

```
  -s, --source <language> Document source language, one of en, zh, ru (default: "en")
```

```
  -t, --target <language> Document target language, one of en, zh, ru (default: "zh")
```

```
  -g, --glob <path...>      Glob patterns of source dirs/files to translate
```

```
  -C, --copy [boolean]      Whether to copy relative assets to the target directory instead of following links (default: false)
```

```
  -h, --help                display help for command
```

- The `-g, --glob` parameter is required. You can specify the directories or files to translate, supporting `glob` syntax. Note that the parameter value must be quoted to avoid unexpected behavior caused by command line parsing. Examples:
 1. `yarn translate -g abc xyz` will translate all documents under `<root>/<source>/abc` and `<root>/<source>/xyz` to `<root>/<target>/abc` and `<root>/<target>/xyz` respectively.
 2. `yarn translate -g '*'` will translate all documents under `<root>/<source>`.
- The `-C, --copy` parameter is optional. It controls whether to copy local resource files to the target directory when the target file does not exist. The default is `false`, which means changing the resource file reference path to the source path. Examples:
 - When enabled:
 1. When translating `/<source>/abc.jpg`, it will copy `<root>/public/<source>/abc.jpg` to `<root>/public/<target>/abc.jpg` and modify the reference path in the document to `/<target>/abc.jpg`.

2. In `<root>/<source>/abc.mdx` , when translating `./assets/xyz.jpg` , it will copy `<root>/<source>/assets/xyz.jpg` to `<root>/<target>/assets/xyz.jpg` , and the image reference path remains unchanged.
 3. In `<root>/<source>/abc.mdx` , when translating `./assets/<source>/xyz.jpg` , it will copy `<root>/<source>/assets/<source>/xyz.jpg` to `<root>/<target>/assets/<target>/xyz.jpg` and modify the reference path in the document to `./assets/<target>/xyz.jpg` .
- When not enabled:
 1. When translating `/<source>/abc.jpg` , if `<root>/public/<target>/abc.jpg` exists, the reference path in the document will be changed to `/<target>/abc.jpg` ; otherwise, the image reference path remains unchanged.
 2. In `<root>/<source>/abc.mdx` , when translating `./assets/<source>/xyz.jpg` , if `<root>/<target>/assets/<target>/xyz.jpg` exists, the reference path will be changed to `./assets/<target>/xyz.jpg` ; otherwise, it will be changed to `../<source>/assets/<target>/xyz.jpg` .

WARNING

Specially, if you use `-g '*'` for full translation, the file lists of `source` and `target` directories will be compared, and unmatched `target` files except for `internalRoutes` will be automatically deleted.

TIP

The translation feature requires configuring the `AZURE_OPENAI_API_KEY` environment variable locally. Please contact your team leader to obtain it.

You can control translation behavior in the document metadata:

```
i18n:  
  title:  
    en: DevOps Connectors  
  additionalPrompts: 'The term Connectors in this document is a proper no  
un and should not be translated'  
  disableAutoTranslation: false  
title: DevOps Connectors
```

Usage

Configuration

Configure the `doom` documentation tool

Configuration File

Basic Configuration

API Documentation Configuration

Permission Documentation Configuration

Reference Documentation Configuration

Release Notes Configuration

Sidebar Configuration

Internal Documentation Routes Configuration

Only Include Documentation Routes Configuration

Syntax Highlighting Plugin Configuration

`sites.yaml` Configuration

Translation Configuration

Editing Documentation in Code Repository

Documentation Export Configuration

Documentation Lint Configuration

Algolia Search Configuration

Sitemap Configuration

Convention

Based on the principle of "convention over configuration", we agree on the organization of documents to automatically generate the left sidebar and related content.

Directory Structure

Metadata

Sorting

Preview

Markdown

Callouts

Mermaid

MDX

Using MDX enables dynamic content display and content reuse

rspress Components

doom Components

Custom Component Reuse

Internationalization

Using Internationalized Text in Reusable Components

`i18n.json`

`.ts/.tsx`

`.mdx`

API Documentation

Advanced APIs

CRD

Common References

Specifying OpenAPI Path

Permission Description Document

[props](#)

Example

Referencing Documents

Document Reference Configuration

Deployment

Configuration

TOC

[Configuration File](#)

Basic Configuration

API Documentation Configuration

Permission Documentation Configuration

Reference Documentation Configuration

`frontmatterMode`

Release Notes Configuration

Sidebar Configuration

Internal Documentation Routes Configuration

Only Include Documentation Routes Configuration

Syntax Highlighting Plugin Configuration

`sites.yaml` Configuration

Translation Configuration

Editing Documentation in Code Repository

Documentation Export Configuration

Documentation Lint Configuration

Algolia Search Configuration

Sitemap Configuration

Configuration File

In most cases, we only need to use a static `yaml` configuration file, supporting `doom.config.yaml` or `doom.config.yml`. For complex scenarios, such as requiring dynamic configuration or custom `rspress` plugins, `js/ts` configuration files can be used, supporting multiple file formats including `.js/.ts/.mjs/.mts/.cjs/.cts`.

For `js/ts` configuration files, we need to export the configuration, which can be combined with the `defineConfig` function exported from `@alauda/doom/config` to provide type assistance:

```
import { defineConfig } from '@alauda/doom/config'

export default defineConfig({})
```

Basic Configuration

- `lang`: Default documentation language. To accommodate most projects, we support both Chinese and English documents by default. The default language is `en`. If the current documentation project does not require multilingual support, this can be set to `null` or `undefined`.
- `title`: Documentation title, displayed on the browser tab.
- `logo`: Logo at the top left of the documentation, supports image URLs or file paths. Absolute paths refer to files under the `public` directory, relative paths refer to files relative to the current tool directory. By default, the built-in Alauda logo from the `doom` package is used.
- `logoText`: Documentation title, displayed next to the logo at the top left.
- `icon`: Documentation favicon, defaults to the same as `logo`.
- `base`: Base path for the documentation, used when deploying to a non-root path, such as `product-docs`. Defaults to `/`.
- `outDir`: Directory for build outputs, defaults to `dist/{base}/{version}`. If specified, it changes to `dist/{outDir}/{version}`, where `version` is optional. See [Multi-version Build](#) for reference.

API Documentation Configuration

```
api:
  # CRD definition file paths, relative to the directory of doom.config.
  *, supports glob matching, json/yaml files
  crds:
    - docs/shared/crds/*.yaml
  # OpenAPI definition file paths, relative to the directory of doom.config.
  *, supports glob matching, json/yaml files
  openapis:
    - docs/shared/openapis/*.json
  # When rendering OpenAPI related resource definitions, they are inlined
  by default. To extract related resource definitions into separate files,
  configure the following options.
  # Reference https://doom.alauda.cn/apis/references/CodeQuality.html#v1alpha1.CodeQualitySpec
  references:
    v1alpha1.CodeQualityBranch: /apis/references/CodeQualityBranch#v1alpha1.CodeQualityBranch
  # Optional, API documentation path prefix. If the current business uses
  gateway or other proxy services, this can be configured.
  pathPrefix: /apis
```

Refer to [API Documentation](#) for writing documentation.

Permission Documentation Configuration

```
# The following resource file paths are relative to the directory of doom.config.
*, support glob matching, json/yaml files
permission:
  functionresources:
    # `kubectl get functionresources`
    - docs/shared/functionresources/*.yaml
  roletemplates:
    # `kubectl get roletemplates -l auth.cpaas.io/roletemplate.official=true`
    - docs/shared/roletemplates/*.yaml
```

Refer to [Permission Documentation](#) for writing documentation.

Reference Documentation Configuration

reference:

- **repo:** `alauda-public/product-doc-guide` # Optional, repository address of the referenced documentation. If not filled, the current documentation repository address is used by default.

- **branch:** # [string] Optional, branch of the referenced documentation repository

- **publicBase:** # [string] Optional, when using a remote repository, the absolute path where static resources corresponding to `/images/xx.png` are located. Defaults to `docs/public`

sources:

- **name:** `anchor` # Name of the referenced documentation, used for referencing in documents, globally unique

- **path:** `docs/index.mdx#□□` # Path of the referenced documentation, supports anchor positioning. For remote repositories, relative to the root of the repository; for local, relative to the directory of `doom.config`.*

- **ignoreHeading:** # [boolean] Optional, whether to ignore the heading. If true, the anchor heading will not be displayed in the referenced document.

- **processors:** # Optional, processors for referenced document content

- **type:** `ejsTemplate`

- **data:** # ejs template parameters, accessed via ``<%= data.xx %>`

- **frontmatterMode:** `merge` # Optional, frontmatter handling mode for referenced documents. Defaults to `ignore`. Options: `ignore/merge/replace/remove`

frontmatterMode

- `ignore`: Ignore the frontmatter of the referenced document, keep using the current document's frontmatter.
- `merge`: Merge the frontmatter of the referenced document. If keys conflict, the referenced document's values override the current document's.

- `replace` : Replace the current document's frontmatter with that of the referenced document.
- `remove` : Remove the current document's frontmatter.

Refer to [Reference Documentation](#) for writing documentation.

Release Notes Configuration

```
releaseNotes:
  queryTemplates:
    fixed: # JQL statements that can include ejs templates
    unfixed:
```

release-notes.md

```
<!-- release-notes-for-bugs?template=fixed&project=DevOps -->
```

release-notes.mdx

```
{/* release-notes-for-bugs?template=fixed&project=DevOps */}
```

Taking the above `template=fixed&project=DevOps` as an example, `fixed` is the template name defined in `queryTemplates`. The remaining `query` parameter `project=DevOps` is passed as `ejs` [↗](#) template parameters to the `fixed` template, which after processing is used to form a jira `jql` [↗](#) request to `https://jira.alauda.cn/rest/api/2/search?jql=<jql>`. This API requires authentication, and the environment variables `JIRA_USERNAME` and `JIRA_PASSWORD` must be provided to preview the effect.

Sidebar Configuration

sidebar:

collapsed: `false` # Optional, whether the sidebar is collapsed by default. Defaults to collapsed. When documentation content is small, consider setting to false.

Internal Documentation Routes Configuration

internalRoutes: # Optional, supports glob matching, relative to the docs directory. When the CLI option `-i, --ignore` is enabled, matched routes/files will be ignored.

- `'*/internal/**'`

Only Include Documentation Routes Configuration

onlyIncludeRoutes: # Optional, supports glob matching, relative to the docs directory. When the CLI option `-i, --ignore` is enabled, only routes/files under this configuration will be enabled. Can be combined with `internalRoutes` to further exclude some routes.

- `'*/internal/**'`

internalRoutes:

- `'*/internal/overview.mdx'`

Syntax Highlighting Plugin Configuration

shiki:

theme: # optional, <https://shiki.style/themes>

langs: # optional, <https://shiki.style/languages>

transformers: # optional, only available in js/ts config, <https://shiki.style/guide/transformers>

WARNING

Unconfigured languages will trigger warnings in the command line and fall back to rendering as `plaintext`.

`sites.yaml` Configuration

The `sites.yaml` configuration file is used to configure subsite information associated with the current documentation site. This information is used by [External Site Components](#) and when building single-version documentation.

```
- name: connectors # Globally unique name
  base: /devops-connectors # Base path for site access
  version: v1.1 # Version used for ExternalSite/ExternalSiteLink redirection when building multi-version sites

  displayName: # Site display name. If not filled or language not matched, defaults to `name`
    en: DevOps Connectors
    zh: DevOps 连接器

  # The following properties are used to pull images when building the entire site. If not filled, this will be ignored during final packaging.
  # Usually required for subsite references, not required for parent site references.
  repo: https://github.com/AlaudaDevops/connectors-operator # Site repository address. For internal gitlab repositories, related slugs like `alauda/product-docs` can be used directly.
  image: devops/connectors-docs # Site build image, used to pull images when building the entire site.
```

Translation Configuration

translate:

```
# System prompt, ejs template. Parameters passed in include `sourceLang`, `targetLang`, `userPrompt`, `additionalPrompts`, `terms`, `titleTranslationPrompt`.
```

```
# Among them, `sourceLang` and `targetLang` are the strings `□□` and `□□`,
```

```
# `userPrompt` is the global user configuration below, which may be empty
```

```
# `additionalPrompts` is the `additionalPrompts` configuration in the document's `frontmatter.i18n`, which may be empty
```

```
# `terms` and `titleTranslationPrompt` are dynamically generated prompts based on terminology and title tables contained in the document, used to guide AI translation according to terminology and title mappings, which may be empty
```

```
# The default system prompt is as follows and can be modified according to actual needs
```

```
systemPrompt: |
```

You are a professional technical documentation engineer, skilled in writing high-quality technical documentation in `<%= targetLang %>`. Please accurately translate the following text from `<%= sourceLang %>` to `<%= targetLang %>`, maintaining the style consistent with technical documentation in `<%= sourceLang %>`.

Baseline Requirements

- Sentences should be fluent and conform to the expression habits of the `<%= targetLang %>` language.
- Input format is MDX; output format must also retain the original MDX format. Do not translate the names of jsx components such as `<Overview />`, and do not wrap output in unnecessary code blocks.
- ****CRITICAL****: Do not translate or modify ANY link content in the document. This includes:
 - **URLs in markdown links**: `[text](URL)` - keep URL exactly as is
 - **Reference-style links**: `[text][ref]` and `[ref]: URL` - keep both ref and URL unchanged
 - **Inline URLs**: `https://example.com` - keep completely unchanged
 - **Image links**: `![alt](src)` - keep src unchanged, but alt text can be translated
 - **Anchor links**: `[text](#anchor)` - keep `#anchor` unchanged
 - Any href attributes in HTML tags - keep unchanged
- **Do not translate professional technical terms and proper nouns, including but not limited to**: Kubernetes, Docker, CLI, API, REST, GraphQL, JSON, YAML, Git, GitHub, GitLab, AWS, Azure, GCP, Linux, Windows, macOS, Node.js, React, Vue, Angular, TypeScript, JavaScript, Python, Java, Go, Rust, e

tc. Keep these terms in their original form.

- The title field and description field in frontmatter should be translated, other frontmatter fields should retain and do not translate.

- Content within MDX components needs to be translated, whereas MDX component names and parameter keys do not.

- Do not modify or translate any placeholders in the format of `__ANCHOR_N__` (where N is a number). These placeholders must be kept exactly as they appear in the source text.

- Keep original escape characters like backslash, angle brackets, etc. unchanged during translation.

- Do not add any escape characters to special characters like `[]`, `()`, `{}`, etc. unless they were explicitly present in the source text. For example:

- If source has "Architecture [Optional]", keep it as "Architecture [Optional]" (not "Architecture `\\[Optional]`")

- If source has "Function (param)", keep it as "Function (param)" (not "Function `\\(param)`")

- Only add escape characters if they were present in the original text

- Preserve and do not translate the following comments, nor modify their content:

- `{/* release-notes-for-bugs */}`

- `<!-- release-notes-for-bugs -->`

- Remove and do not retain the following comments:

- `{/* reference-start */}`

- `{/* reference-end */}`

- `<!-- reference-start -->`

- `<!-- reference-end -->`

- Ensure the original Markdown format remains intact during translation, such as frontmatter, code blocks, lists, tables, etc.

- Do not translate the content of the code block.

```
<% if (titleTranslationPrompt) { %>
```

```
<%- titleTranslationPrompt %>
```

```
<% } %>
```

```
<% if (terms) { %>
```

```
<%- terms %>
```

```
<% } %>
```

```
<% if (userPrompt || additionalPrompts) { %>
```

```
## Additional Requirements
```

These are additional requirements for the translation. They should be met along with the baseline requirements, and in case of any conflict, the baseline requirements should take precedence.

The text for translation is provided below, within triple quotes:

```
"""
```

```
<% if (userPrompt) { %>
<%- userPrompt %>
<% } %>

<% if (additionalPrompts) { %>
<%- additionalPrompts %>
<% } %>
""
<% } %>
```

Editing Documentation in Code Repository

editRepoBaseUrl: `alauda/doom/tree/main/docs` # The `https://github.com/` prefix can be omitted. Only effective when the CLI flag ``-R, --edit-repo`` is enabled.

Documentation Export Configuration

export:

- **name:** `Concepts` # Optional, globally unique PDF name, defaults to the

Convention

TOC

[Directory Structure](#)[Metadata](#)[Sorting](#)[Preview](#)

Directory Structure

The left sidebar is automatically generated based on the file directory structure, where the `index` file in the first-level directory acts as the document's homepage and will display as the first item in the left navigation. Subfolders can use `index.md` or `index.mdx` and define the first-level title to set the grouping title for the left sidebar. Other sub-documents will be automatically merged into the current group, and nested subfolders will follow the same rules.

```
├─ index.md
├─ start.mdx
└─ usage
    ├─ index.mdx
    └─ convention.md
```

We also agree that:

1. The `public` directory is used to store static resources such as images, videos, etc.

2. The `public/_remotes` directory is used to store static resources associated with [remote reference documents](#). Please do not directly rely on resources from this directory; you may add `*/public/_remotes` to `.gitignore` to prevent these from being committed to the code repository.
3. The `shared` directory is for storing common components, reusable documents, etc., and will not automatically generate document data.

Metadata

At the beginning of the document, you can define the document's metadata such as title, description, author, category, etc., through the `frontmatter`.

```
---  
title: Title  
description: Description  
author: Author  
category: Category  
---
```

In the body of the document, when using `.mdx` files, you can access these metadata through `frontmatter` as described in [MDX](#).

Sorting

Other documents, except for `index.md` or `index.mdx`, will be sorted by default according to their file names. You can customize the `weight` value in the `frontmatter` to adjust the order of documents in the left sidebar (the smaller the `weight` value, the higher the priority in sorting).

```
---  
weight: 1  
---
```

Warning

Note: Currently, changes to the left navigation configuration require a service restart to take effect, and it is usually not necessary to pay too much attention during development.

Preview

Sometimes, we do not need to display special content on the group homepage. In this case, you can use `index.mdx` file and the `Overview` component to display the list of documents in the current group. This will showcase the titles, descriptions, and secondary title information of the grouped list file.

Usage

```
<Overview />
```

You can refer to [Usage](#) for the effect.

Markdown

In addition to the standard [gfm](#) [↗] syntax, Doom has built-in some extra Markdown extension features.

TOC

Callouts

Mermaid

Callouts

Source code annotation components

Note

1. Please use inline code comments according to the actual language, such as `;`, `%`, `#`, `//`, `/** */`, `--`, and `<!-- -->`, etc.
2. If you want to treat them as code comments, please escape them with `[\!code callout]`.
3. Sometimes, `:::callouts` may display abnormally due to nested indentation; you can use `<div class="doom-callouts">` or the `<Callouts>` component instead.

```

```sh
Memory overhead per virtual machine ≈ (1.002 × requested memory) \
 + 218 MiB \ # [!code callout]
 + 8 MiB × (number of vCPUs) \ # [!code callout]
 + 16 MiB × (number of graphics devices) \ # [!code callout]
t]
 + (additional memory overhead) # [!code callout]
...

```

:::callouts

1. Required for the processes that run in the `virt-launcher` pod.
2. Number of virtual CPUs requested by the virtual machine.
3. Number of virtual graphics cards requested by the virtual machine.
4. Additional memory overhead:
  - If your environment includes a Single Root I/O Virtualization (SR-IOV) network device or a Graphics Processing Unit (GPU), allocate 1 GiB additional memory overhead for each device.
  - If Secure Encrypted Virtualization (SEV) is enabled, add 256 MiB.
  - If Trusted Platform Module (TPM) is enabled, add 53 MiB.

:::

```

Memory overhead per virtual machine ≈ (1.002 × requested memory) \
 + 218 MiB \ ①
 + 8 MiB × (number of vCPUs) \ ②
 + 16 MiB × (number of graphics devices) \ ③
 + (additional memory overhead) ④

```

1. Required for the processes that run in the `virt-launcher` pod.
2. Number of virtual CPUs requested by the virtual machine.
3. Number of virtual graphics cards requested by the virtual machine.
4. Additional memory overhead:
  - If your environment includes a Single Root I/O Virtualization (SR-IOV) network device or a Graphics Processing Unit (GPU), allocate 1 GiB additional memory overhead for each device.
  - If Secure Encrypted Virtualization (SEV) is enabled, add 256 MiB.

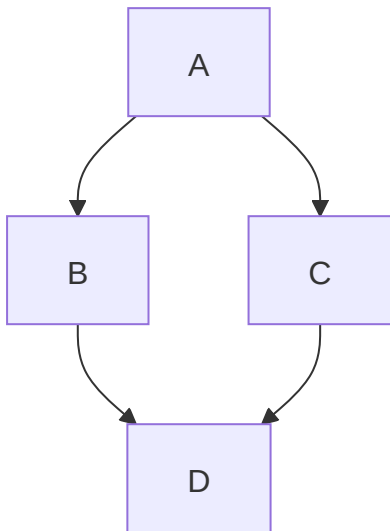
- If Trusted Platform Module (TPM) is enabled, add 53 MiB.

For more source code transformation features, please refer to [Shiki Transformers](#) ↗.

## Mermaid ↗

Chart drawing tool

```
```mermaid
graph TD;
  A-->B;
  A-->C;
  B-->D;
  C-->D;
```
```



# MDX

[MDX ↗](#) is an extension syntax of Markdown that allows the use of JSX syntax within Markdown. For usage, refer to [rspress MDX ↗](#).

## TOC

### [rspress Components](#)

#### doom Components

[Overview](#)[Directive](#)[ExternalSite](#)[ExternalSiteLink](#)[AcpApisOverview](#) and [ExternalApisOverview](#)

#### Term

[props](#)[TermsTable](#)[props](#)[JsonViewer](#)

#### Custom Component Reuse

## rspress Components

Most of the [built-in components](#) provided by the `rspress` theme have been adjusted to global components, which can be used directly in `.mdx` files without import, including:

- `Badge`
- `Card`
- `LinkCard`
- `PackageManagerTabs`
- `Steps`
- `Tab/Tabs`
- `Toc`

Other less commonly used components can be imported from `rspress/theme`, for example:

preview.mdx

```
import { SourceCode } from '@rspress/core/theme'

<SourceCode href="/" />
```

## doom Components

`doom` provides some global components to assist in documentation writing, which can be used directly without import. Currently, these include:

### Overview

Document overview component used to display the document directory

### Directive

Sometimes, due to nested indentation, the [custom container](#) syntax may fail. You can use the `Directive` component as a replacement.

- The directory structure of multilingual documents (``doc/en``) needs to be exactly the same as the documents under the ``doc/zh`` directory to ensure that the links in multilingual documents are identical except for the language identifier.

```
<Directive type="danger" title="Note">
```

If you are using automated translation tools, you do not need to worry about

this issue. The automated translation tools will automatically generate the

target language document directory structure based on ``doc/zh``.

```
</Directive>
```

- The directory structure of multilingual documents ( `doc/en` ) needs to be exactly the same as the documents under the `doc/zh` directory to ensure that the links in multilingual documents are identical except for the language identifier.

#### Note

If you are using automated translation tools, you do not need to worry about this issue. The automated translation tools will automatically generate the target language document directory structure based on `doc/zh`.

## ExternalSite

Component for referencing external sites

```
<ExternalSite name="connectors" />
```

#### Note

Because DevOps Connectors releases on a different cadence from Alauda Container Platform, the DevOps Connectors documentation is now available as a separate documentation set at [DevOps Connectors](#).

## ExternalSiteLink

Component for referencing external site links

```
<ExternalSiteLink name="connectors" href="link.mdx#hash" children="Content" />
```

[Content ↗](#)

### TIP

In mdx, `<ExternalSiteLink name="connectors" href="link" children="Content" />` has a different meaning from the following content:

```
<ExternalSiteLink name="connectors" href="link">
 Content {/* will be rendered inside a `p` element */}
</ExternalSiteLink>
```

If you do not want the text to be rendered inside a `p` element, you can pass it using the `children` attribute as shown in the example above.

## AcpApisOverview and ExternalApisOverview

Components for referencing external site API overviews

```
<AcpApisOverview />
{/* same as following */}
<ExternalApisOverview name="acp" />

<ExternalApisOverview name="connectors" />
```

Note

For the introduction to the usage methods of ACP APIs, please refer to [ACP APIs Guide ↗](#).

Note

For the introduction to the usage methods of DevOps Connectors APIs, please refer to [DevOps Connectors APIs Guide ↗](#).

## Term

Term component, plain text, dynamically mounted and injected

```
<Term name="company" textCase="capitalize" />
<Term name="product" textCase="lower" />
<Term name="productShort" textCase="upper" />
<Term name="alaudaCloudLink" />
```

Alauda alauda container platform ACP [Alauda Cloud](#) ↗

props

- `name`: Built-in term name, refer to [dynamic mount configuration file](#)
- `textCase`: Text case transformation, optional values are `lower`, `upper`, `capitalize`

TermsTable

Built-in term list display component

```
<TermsTable />
```

Name	Chinese	Chinese Bad Cases	English	English Bad Cases	Description
company	公司	-	Alauda	-	公司
product	容器平台 ACP	-	Alauda Container Platform	-	容器平台
productShort	ACP	-	ACP	-	容器平台



Name	Chinese	Chinese Bad Cases	English	English Bad Cases	Description
alaudaCloudLink	<a href="#">Alauda Cloud ↗</a>	-	<a href="#">Alauda Cloud ↗</a>	-	-

### props

- `terms`: `NormalizedTermItem[]`, optional, custom term list for convenient reuse when rendering custom terms in internal documentation

### JsonViewer

```
<JsonViewer value={{ key: 'value' }} />
```

yaml json

```
key: value
```

## Custom Component Reuse

According to the [convention](#), we can extract reusable content into the `shared` directory, then import it where needed, for example:

```
import CommonContent from './shared/CommonContent.mdx'

<CommonContent />
```

If you need to use more [runtime](#) related APIs, you can implement components with `.jsx/.tsx` and then import and use them in `.mdx` files.

```
// shared/CommonContent.tsx
export const CommonContent = () => {
 const { page } = usePageData()
 return <div>{page.title}</div>
}

// showcase/content.mdx
import { CommonContent } from './shared/CommonContent'
;<CommonContent />
```

# Internationalization

Most of the internal documentation of `alauda` is bilingual in Chinese and English. Therefore, we by default support using `en` / `zh` subfolders to store documents in different languages. It is recommended to also store static resources under `en` / `zh` subfolders in the `public` directory, which facilitates the management of both documentation content and static resources.

## TOC

`i18n.json`

`.ts/.tsx`

`.mdx`

### `i18n.json`

For reusable components, if you need to support both Chinese and English within the same component, you first need to create an `i18n.json` file under the `docs` directory. Then, in the component, use `useI18n` to get the text for the current language, for example:

```
docs/i18n.json
```

```
{
 "title": {
 "zh": "标题",
 "en": "Title"
 },
 "description": {
 "zh": "描述",
 "en": "description"
 }
}
```

## .ts/.tsx

```
import { useI18n } from '@rspress/core/runtime'

export const CommonContent = () => {
 const t = useI18n()
 return <h1>{t('title')}</h1>
}
```

## .mdx

```
import { useI18n } from '@rspress/core/runtime'

{useI18n()('title')}
```

# API Documentation

Based on actual business needs, we generally categorize APIs into two types: Advanced APIs and CRDs (Custom Resource Definitions). Therefore, the directory structure is typically organized as follows:

```
|— apis
| |— advanced-apis # Advanced APIs
| |— crds # CRDs
| |— references # Common References
```

## TOC

### Advanced APIs

[props](#)

### CRD

[props](#)

### Common References

[props](#)

### Specifying OpenAPI Path

## Advanced APIs

advanced-apis/codeQualityTaskSummary.mdx

```
CodeQualityTaskSummary
```

```
<OpenAPIPath path="/plugins/v1alpha1/template/codeQuality/task/{task-id}/summ
```

Refer to [CodeQualityTaskSummary](#).

### props

- `path`: The path under OpenAPI schema `paths`
- `pathPrefix`: Can be used to override the `api.pathPrefix` in global configuration
- `openapiPath`: Refer to [Specifying OpenAPI Path](#)

## CRD

```
crds/ArtifactCleanupRun.mdx
```

```
ArtifactCleanupRun
```

```
<K8sCrd name="artifactcleanupruns.artifacts.katanomi.dev" />
```

Refer to [ArtifactCleanupRun](#).

### props

- `name`: CRD `metadata.name`
- `crdPath`: Similar to [Specifying OpenAPI Path](#), used to specify a particular CRD file

## Common References

```
references/CodeQuality.mdx
```

```
CodeQuality
```

```
<OpenAPIRef schema="v1alpha1.CodeQuality" />
```

Refer to [CodeQuality](#).

## props

- `schema` : The name under OpenAPI schema `definitions` (v2) or `components/schemas` (v3)
- `openapiPath` : Refer to [Specifying OpenAPI Path](#)

## Specifying OpenAPI Path

For the `OpenAPIPath` and `OpenAPIRef` components, the default behavior is to search for matches across all OpenAPI definition files. If you need to specify a particular OpenAPI file, you can use the `openapiPath` property:

```
<OpenAPIPath
 path="/plugins/v1alpha1/template/codeQuality/task/{task-id}/summary"
 openapiPath="shared/openapis/katanomi.json"
>
```

# Permission Description Document

```
<K8sPermissionTable functions={['devops-testplans', 'devops-testmodules']} />
```

## TOC

[props](#)[Example](#)

### props

- `functions`: `string[]` - Required. An array of `FunctionResource` resource names to be displayed.

## Example

Function	Action	Platform Administrator	Platform auditors	Project Manager	Namespace Administrator
testplans	View	✓	✓	✓	✓
devops-testplans	Create	✓	✗	✓	✓



Function	Action	Platform Administrator	Platform auditors	Project Manager	Namespace Administrator
testmodules   devops-   testmodules	Update	✓	×	✓	✓
	Delete	✓	×	✓	✓
	View	✓	✓	✓	✓
	Create	✓	×	✓	✓
	Update	✓	×	✓	✓

# Referencing Documents

In Markdown files:

```
<!-- reference-start#name -->

<!-- reference-end -->
```

In MDX files:

```
{/* reference-start#name */}

{/* reference-end */}
```

The `name` above refers to the name of the referenced document. For more information, please refer to [Document Reference Configuration](#). If the referenced document content uses static resources from a remote repository, the related static resources will be automatically stored locally in the `<root>/public/_remotes/<name>` directory.

Here is an example using `<!-- reference-start#ref -->`:

---

## TOC

Document Reference Configuration

`frontmatterMode`

---

# Document Reference Configuration

## reference:

- **repo:** `alauda-public/product-doc-guide` # Optional, repository address for the referenced document. If not provided, the current document repository address will be used by default.

- **branch:** # [string] Optional, branch of the referenced document repository.

- **publicBase:** # [string] Optional, the directory where static resources for remote repository located, corresponding to absolute paths like `/images/xx.png`. Default is `docs/public`.

## sources:

- **name:** `anchor` # Name of the referenced document, used to reference within the document and must be globally unique.

- **path:** `docs/index.mdx#introduction` # Path to the referenced document, supports anchor targeting; for remote repositories, relative to the repository root directory, and for local, relative to the directory of `doom.config.*`.

- **ignoreHeading:** # [boolean] Optional, whether to ignore headings. If true, the anchor's title will not be displayed in the referenced document.

- **processors:** # Optional, processors for handling the content of the referenced document.

- **type:** `ejsTemplate`

- **data:** # EJS template parameters, accessed via `<%= data.xx %>`.

- **frontmatterMode:** `merge` # Optional, mode for handling the frontmatter of the referenced document. Default is `ignore`. Possible values are `ignore/merge/replace/remove`.

## frontmatterMode

- **ignore**: Ignores the frontmatter of the referenced document and retains the frontmatter of the current document.
- **merge**: Merges the frontmatter of the referenced document. If there are the same keys, the values from the referenced document will overwrite those in the current document.
- **replace**: Replaces the frontmatter of the current document with that of the referenced document.

- `remove` : Removes the frontmatter of the current document.

For writing documentation, refer to [Document Reference](#).

# Deployment

---

## TOC

### Build and Preview

Multi-version Build

Merged Directory Structure

Dynamic Mount Configuration File

---

## Build and Preview

Before deployment, we need to build the project for production and preview it locally to ensure the project runs correctly:

```
doom build # Build static assets
doom serve # Preview the build assets in production mode
```

## Multi-version Build

By default, `doom build` outputs the build artifacts to the `dist` directory. If you need to build multiple versions of the documentation, you can specify the version number with the `-v` parameter, for example:

```
Usually determined by branch name, e.g., release-4.0 corresponds to version 4.0
doom build -v 4.0 # Build version 4.0, output to dist/4.0, documentation access path is {base}/4.0
doom build -v master # Build master version, output to dist/master, documentation access path is {base}/master
doom build -v {other} # Build other versions, output to dist/{other}, documentation access path is {base}/{other}

unversioned and unversioned-x.y are special version numbers used to build documentation without version prefix
doom build -v unversioned # Build documentation without version prefix, output to dist/unversioned, documentation access path is {base}
doom build -v unversioned-4.0 # Build documentation without version prefix but showing version 4.0 in the navbar, output to dist/unversioned, documentation access path is {base}
```

## Merged Directory Structure

```
|— console-platform
| |— 4.0
| |— 4.1
| |— index.html
| |— overrides.yaml
| └— versions.yaml
|— console-devops-docs
| |— 4.0
| |— 4.1
| |— index.html
| |— overrides.yaml
| └— versions.yaml
|— console-tekton-docs
| |— 1.0
| |— 1.1
| |— index.html
| |— overrides.yaml
| └— versions.yaml
```

index.html

```
<!DOCTYPE html>
<html>
 <head>
 <title>Redirecting...</title>
 <meta http-equiv="refresh" content="0; url=/console-docs/4.1" />
 </head>
 <body>
 <p>Redirecting to /console-docs/4.1</p>
 </body>
</html>
```

## Dynamic Mount Configuration File

overrides.yaml

```
Documentation information, each document can mount to override default conf
title:
 en: Doom - Alauda
 zh: Doom - 方块
logoText:
 en: Doom - Alauda
 zh: Doom - 方块
```

versions.yaml

```
- '4.1'
- '4.0'
```